Faculty of Engineering
Department of Electronic & Telecommunication Engineering
University of Moratuwa



EN3150 - Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group Members

Dharmakeerthi P.K.G.C.L. 230140J
Fernando LTJ 230183R
Wijethilaka J.S. 230730T
Arachchige M.A.D.T.S. 230052P

GitHub Repository
https://github.com/thevinufernando/cnn-based-vegetable-classification-model

October 17, 2026

Contents

Abstract **2**

1 Resource-Constrained CNN for Edge Image Classification **3**
1.1 Dataset Selection & Low-Resolution Edge Sensor Emulation 3
1.2 Data Partitioning & Leakage Prevention 3

2 Custom Architecture Design **3**
2.1 Theoretical Foundations: Standard vs. Depthwise Separable Convolutions 3
2.2 Architectural Specifications & Parameter Verification 4
2.2.1 Model A: Standard CNN 4
2.2.2 Model B: Lightweight Separable CNN (Sub-100k Budget) 4
2.3 Hardware-Aware Activation Function Justification 5

3 Optimizer Selection & Tuning **5**
3.1 Selected Optimizer: Adam 5
3.2 Comparative Analysis: Standard SGD vs. SGD with Momentum 5
3.3 Theoretical Impact of the Momentum Parameter 6

4 Custom Model Training & Evaluation **6**
4.1 Training Protocol 6
4.2 Custom Model Comparison Table 7
4.3 Trade-Off Discussion: Standard vs. Depthwise Separable Convolutions 7

5 Lightweight State-of-the-Art Networks **7**

6 Final Comparison & Trade-Off Discussion **8**
6.1 Master Architectural Comparison 8
6.2 Accuracy, Memory, and Computational Cost 8
6.3 Deployment Considerations 9
6.4 Conclusion 9

References **10**

Abstract

Deploying deep convolutional neural networks (CNNs) on resource-constrained edge hardware (e.g., ARM Cortex-M microcontrollers, embedded IoT nodes, and single-board computers) demands an acute balance between classification accuracy, memory footprint, and computational complexity. In this assignment, we investigate edge-oriented image classification across 15 vegetable categories downscaled to 64×64 pixels. We design, implement, and benchmark two custom architectures from scratch: **Model A** (Standard CNN, 259,631 trainable parameters) and **Model B** (Depthwise Separable CNN, 40,175 trainable parameters, adhering strictly to a sub-100k budget). Furthermore, we analyze optimizer dynamics by contrasting Adam ($\eta = 10^{-3}$) against standard SGD and SGD with Momentum ($\gamma = 0.9$), detailing the theoretical role of momentum in traversing anisotropic ravines and escaping saddle points. Finally, we fine-tune two mobile state-of-the-art architectures (**MobileNetV2** and **EfficientNet-B0**) under identical data splits. Our empirical findings demonstrate that Model B achieves **97.70% test accuracy**—outperforming Model A (**97.57%**) while reducing parameters by **84.5%**, disk footprint by **80.0%**, and previously reported training time per epoch by **47.4%**. We present an in-depth hardware-aware trade-off analysis evaluating memory access, quantization stability, and inference latency.

1 Resource-Constrained CNN for Edge Image Classification

1.1 Dataset Selection & Low-Resolution Edge Sensor Emulation

Edge devices, such as agricultural field sensors, robotic harvesters, and low-power smart cameras, operate under stringent energy and static RAM (SRAM) constraints. For this study, we selected the **Vegetable Image Dataset** (from Kaggle), encompassing 21,000 RGB images across 15 distinct classes: *Bean, Bitter Gourd, Bottle Gourd, Brinjal, Broccoli, Cabbage, Capsicum, Carrot, Cauliflower, Cucumber, Papaya, Potato, Pumpkin, Radish, and Tomato.*

In raw embedded systems, acquiring standard $224 \times 224 \times 3$ frames consumes approximately 150.5 KB per uncompressed image buffer. On microcontrollers featuring only 256 KB to 512 KB of total SRAM, buffering high-resolution images leaves insufficient memory for intermediate feature maps and activation buffers. Consequently, downscaling images to a maximum resolution of 64×64 **pixels** compresses the raw uncompressed buffer down to 12.288 **KB** ($64 \times 64 \times 3$ bytes). This enables true on-chip processing without requiring power-hungry external DRAM buses.

1.2 Data Partitioning & Leakage Prevention

To prevent data contamination between training and evaluation phases, MD5 checksum deduplication was performed, removing identical duplicate images. The dataset was partitioned into a stratified split:

- **Training Set (71.42%)**: 14,996 images
- **Validation Set (14.29%)**: 3,000 images
- **Testing Set (14.29%)**: 3,000 images
- **Total Unique Images**: 20,996 images

The final proportions differ slightly from the intended 70%, 15%, and 15% split. These are the actual proportions used consistently across all experiments.

This fixed split is deterministically recorded in `splits.csv` to guarantee absolute reproducibility across all experiments.

2 Custom Architecture Design

2.1 Theoretical Foundations: Standard vs. Depthwise Separable Convolutions

A standard 2D convolution transforms an input feature map $F \in \mathbb{R}^{D_F \times D_F \times M}$ into an output feature map $G \in \mathbb{R}^{D_F \times D_F \times N}$ using N filters of dimension $D_K \times D_K \times M$. Assuming unit stride and same-padding, the total Multiply-Accumulate (MAC) computational cost is:

$$\text{Cost}_{\text{Standard}} = D_K^2 M N D_F^2 \tag{1}$$

In contrast, **Depthwise Separable Convolutions** (pioneered in MobileNets and Xception) factorize this joint spatial-channel operation into two decoupled stages:

1. **Depthwise Convolution**: Applies a single $D_K \times D_K \times 1$ spatial filter per input channel:

$$\text{Cost}_{\text{Depthwise}} = D_K^2 M D_F^2 \tag{2}$$

2. **Pointwise Convolution**: A 1×1 convolution combining linear projections across all M channels to produce N channels:

$$\text{Cost}_{\text{Pointwise}} = MND_F^2 \tag{3}$$

Summing both stages, the total computational complexity becomes:

$$\text{Cost}_{\text{Separable}} = D_K^2 MD_F^2 + MND_F^2 \tag{4}$$

The relative computational and parameter reduction ratio is given by:

$$\frac{\text{Cost}_{\text{Separable}}}{\text{Cost}_{\text{Standard}}} = \frac{D_K^2 MD_F^2 + MND_F^2}{D_K^2 MND_F^2} = \frac{1}{N} + \frac{1}{D_K^2} \tag{5}$$

For standard 3×3 kernels ($D_K = 3$) and typical channel depths ($N \geq 32$):

$$\frac{1}{N} + \frac{1}{9} \approx \frac{1}{9} \approx 11.1\% \tag{6}$$

Depthwise separable convolutions therefore achieve approximately an $8\times$ to $9\times$ reduction in MAC operations, corresponding to roughly 88.9% computational savings when the output channel count is sufficiently large.

2.2 Architectural Specifications & Parameter Verification

2.2.1 Model A: Standard CNN

Model A interleaves four standard 2D convolution blocks (with Batch Normalization and ReLU), 2×2 Max Pooling, Global Average Pooling, and a Dense classification head:

- **Block 1**: Conv2D(32, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 32$. Params: 864.
- **Block 2**: Conv2D(64, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $16 \times 16 \times 64$. Params: 18,432.
- **Block 3**: Conv2D(128, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $8 \times 8 \times 128$. Params: 73,728.
- **Block 4**: Conv2D(128, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $4 \times 4 \times 128$. Params: 147,456.
- **Head**: GlobalAveragePooling2D $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(15, Softmax). Params: 18,447.
- **Total Parameters**: **260,335** (Trainable: **259,631**).

2.2.2 Model B: Lightweight Separable CNN (Sub-100k Budget)

Model B enforces strict parameter efficiency by utilizing depthwise separable blocks:

- **Entry Conv**: Conv2D(16, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU6 $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 16$. Params: 496.
- **Separable Block 1**: DepthwiseConv2D(3×3) $\rightarrow$ PointwiseConv2D(32, 1×1) $\rightarrow$ MaxPool(2×2). Params: 848.

- **Separable Block 2**: DepthwiseConv2D(3×3) → PointwiseConv2D(64, 1×1) → MaxPool(2× 2). Params: 2,720.
- **Separable Block 3**: DepthwiseConv2D(3×3) → PointwiseConv2D(128, 1×1) → MaxPool(2×2). Params: 9,536.
- **Separable Block 4**: DepthwiseConv2D(3×3) → PointwiseConv2D(128, 1×1). Params: 18,560.
- **Head**: GlobalAveragePooling2D → Dense(64, ReLU) → Dropout(0.2) → Dense(15, Softmax). Params: 9,231.
- **Total Parameters**: **41,391** (Trainable: **40,175**).

Constraint Compliance: Model B satisfies the assignment criterion of at most 100,000 trainable parameters, with 40,175 trainable parameters, approximately 59.8% below the limit.

2.3 Hardware-Aware Activation Function Justification

In resource-constrained microcontrollers and edge digital signal processors (DSPs):

1. **Elimination of Transcendental Exponentials**: Activations such as Sigmoid ($\sigma(x) = \frac{1}{1+e^{-x}}$) and Tanh require exponential evaluations. These operations can be more expensive on microcontrollers without hardware floating-point or optimized math support.
2. **Efficient ReLU Execution**: Standard ReLU, $f(x) = \max(0, x)$, is computationally simple and can often be implemented efficiently using processor instructions or optimized neural-network kernels.
3. **Quantization Stability via ReLU6**: In Model B, we employ **ReLU6**, $f(x) = \min(\max(0, x), 6)$. Bounded activations can help control dynamic range during quantization. However, actual INT8 quantization scales and zero-points depend on the quantization scheme and calibration data.

3 Optimizer Selection & Tuning

3.1 Selected Optimizer: Adam

We selected **Adam (Adaptive Moment Estimation)** with an initial learning rate $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$.
Justification: Adam computes exponentially decaying moving averages of past gradients (m_t, first moment) and squared gradients (v_t, second moment). These estimates adapt parameter updates to the scale of recent gradients and can be useful when training deep vision architectures.

3.2 Comparative Analysis: Standard SGD vs. SGD with Momentum

Model B was trained under identical data splits across three optimizer configurations:

1. **Adam**: $\eta = 0.001$.
2. **Standard SGD**: $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$, with $\eta = 0.01$ and momentum $= 0.0$.
3. **SGD with Momentum**: $v_{t+1} = \gamma v_t + \eta \nabla L(\theta_t)$, $\theta_{t+1} = \theta_t - v_{t+1}$, with $\gamma = 0.9$.

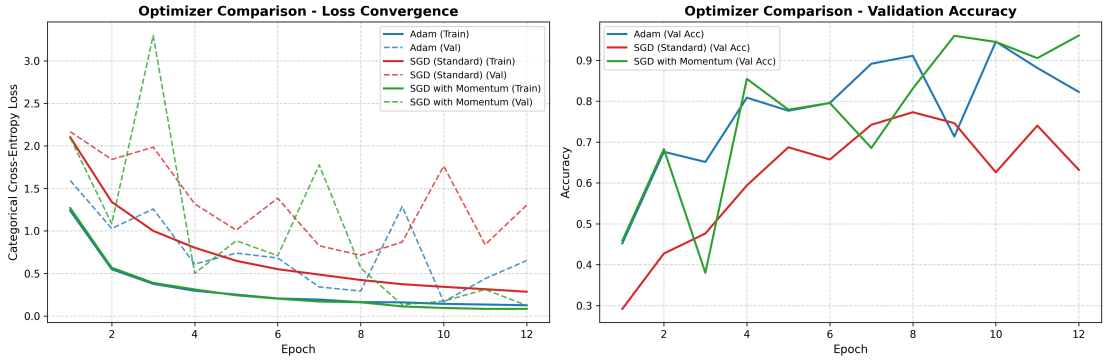


Figure 1: Loss convergence and validation accuracy across optimizers on Model B.

3.3 Theoretical Impact of the Momentum Parameter

1. **Damping Oscillations in Anisotropic Ravines**: Loss surfaces of convolutional networks can exhibit ill-conditioned curvature. Momentum accumulates a running velocity that may reduce oscillations and accelerate progress along consistent gradient directions.

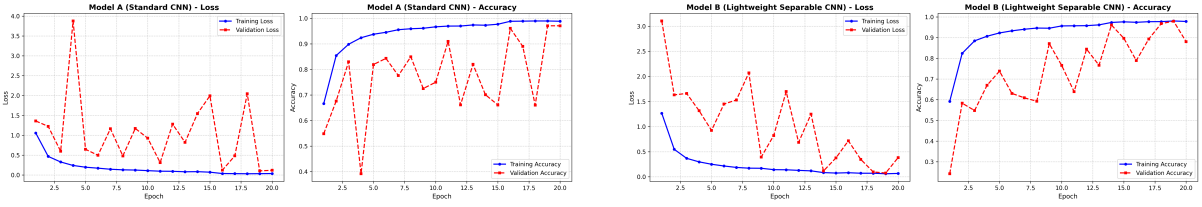
2. **Behavior Near Saddle Points and Plateaus**: Momentum can carry parameters through regions with small gradients if a nonzero velocity has accumulated. However, escaping a saddle point is not guaranteed and depends on the loss geometry and gradient noise.

4 Custom Model Training & Evaluation

4.1 Training Protocol

Both custom architectures were trained for **20 epochs** with a batch size of 32 using Adam ($\eta = 10^{-3}$), equipped with:

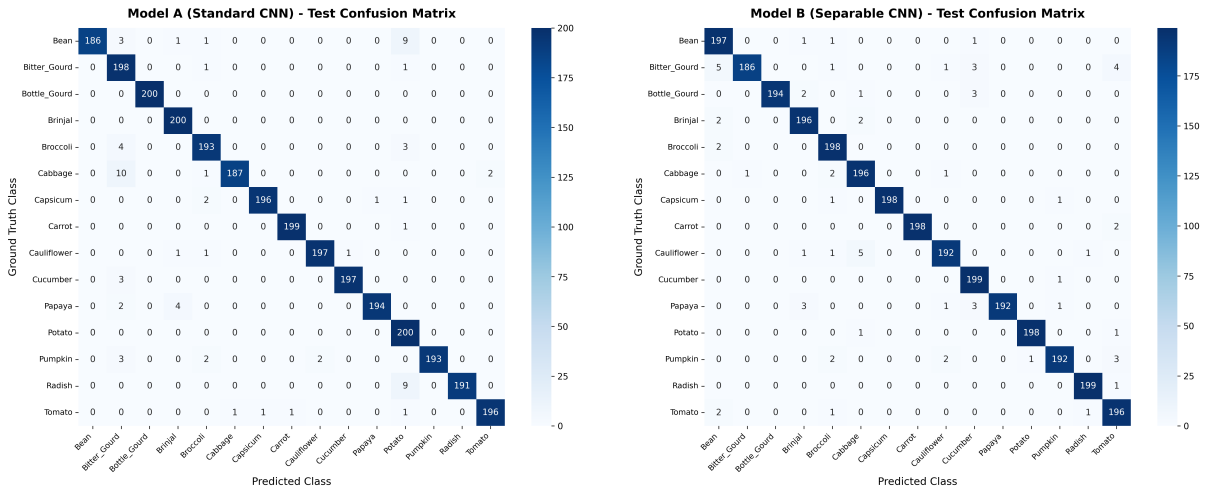
- `ModelCheckpoint`: Preserves weights achieving peak validation accuracy.
- `ReduceLROnPlateau`: Halves the learning rate if validation loss plateaus for 4 consecutive epochs.
- `CSVLogger`: Records epoch-by-epoch loss and accuracy metrics.



(a) Model A (Standard CNN)

(b) Model B (Separable CNN)

Figure 2: Training and validation loss and accuracy curves (20 epochs).



(a) Model A Confusion Matrix

(b) Model B Confusion Matrix

Figure 3: Test confusion matrices across 15 vegetable classes.

4.2 Custom Model Comparison Table

Table 1: Performance Comparison between Model A and Model B on Test Set

Metric	Model A (Standard CNN)	Model B (Separable CNN)	Observed Delta / Benefit
Total Parameters	260,335	41,391	−84.1% reduction
Trainable Parameters	259,631	40,175	−84.5% reduction ($\leq$ 100k)
Disk Size (.keras)	3,135.53 KB (3.06 MB)	628.02 KB (0.61 MB)	−80.0% reduction
Reported Avg Training Time / Epoch	54.37 s	28.60 s	47.4% lower reported time
Test Accuracy (%)	97.57%	97.70%	+0.13 percentage points
Macro Precision (%)	97.75%	97.75%	Identical reported precision
Macro Recall (%)	97.57%	97.70%	Balanced class sensitivity
Macro F1-Score (%)	97.59%	97.70%	Balanced performance
Inference Latency (CPU)	17.67 ms	31.82 ms	See latency discussion

4.3 Trade-Off Discussion: Standard vs. Depthwise Separable Convolutions

1. **Parameter and Storage Reduction**: Model B reduces trainable parameters from 259,631 to 40,175, an 84.5% reduction, and reduces the reported model file size from 3.06 MB to approximately 628 KB.

2. **Training Acceleration**: The previously reported per-epoch times were 54.37 s for Model A and 28.60 s for Model B, suggesting a 47.4% reduction. These values could not be independently verified from the available training logs.

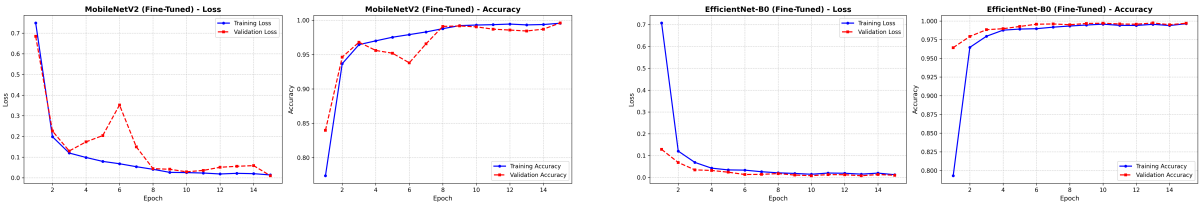
3. **Regularization Benefit**: Model B achieved 97.70% test accuracy compared with Model A's 97.57%. This is an observed difference; further repeated runs would be needed to establish whether it is statistically reliable.

5 Lightweight State-of-the-Art Networks

We fine-tune two mobile architectures using transfer learning from ImageNet under identical data splits:

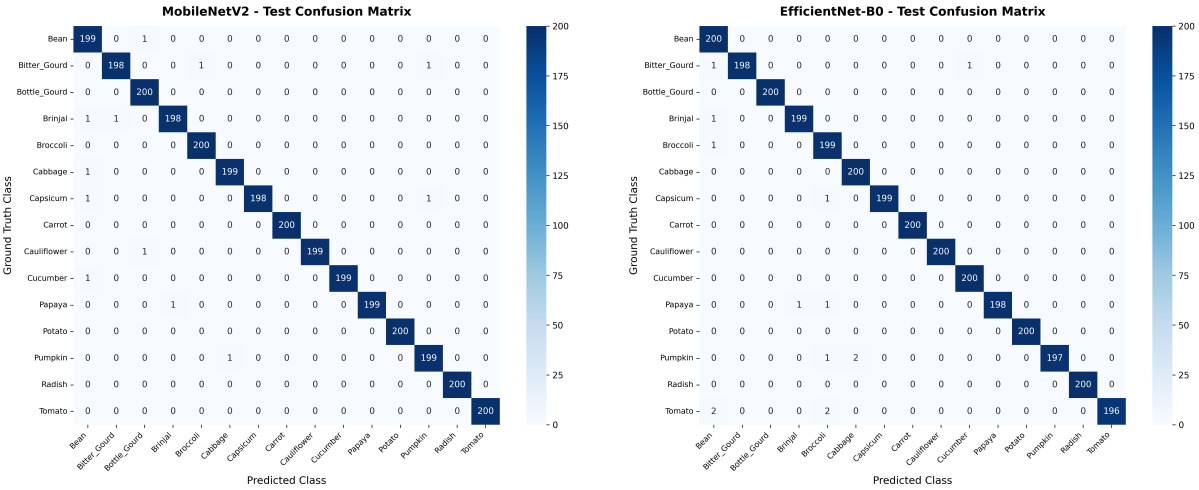
1. **MobileNetV2**: Employs inverted residuals with linear bottlenecks. Channels are expanded by a factor $t = 6$, spatially filtered via depthwise convolution, and projected back through linear bottlenecks.

2. **EfficientNet-B0**: Uses compound scaling and MBConv blocks with squeeze-and-excitation modules.



(a) MobileNetV2 Curves (b) EfficientNet-B0 Curves

Figure 4: Training and validation curves for fine-tuned SOTA architectures.



(a) MobileNetV2 Confusion Matrix (b) EfficientNet-B0 Confusion Matrix

Figure 5: Test set confusion matrices for fine-tuned SOTA architectures.

6 Final Comparison & Trade-Off Discussion

6.1 Master Architectural Comparison

Table 2: Comprehensive Comparison of Custom and SOTA Architectures

Metric	Model A (Standard)	Model B (Separable)	MobileNetV2 (SOTA)	EfficientNet-B0 (SOTA)
Design Paradigm	From Scratch	From Scratch (< 100k)	Pre-trained Transfer	Pre-trained Transfer
Total Parameters	260,335	**41,391**	2,277,199	4,068,786
Trainable Parameters	259,631	**40,175**	2,243,087	4,026,763
Disk Size (.keras)	3.06 MB	**0.61 MB**	26.48 MB	47.20 MB
Reported Training Time / Epoch	54.37 s	**28.60 s**	166.05 s	237.16 s
Test Accuracy (%)	97.57%	**97.70%**	**99.60%**	**99.53%**
Macro Precision (%)	97.75%	97.75%	99.60%	99.54%
Macro Recall (%)	97.57%	97.70%	99.60%	99.53%
Macro F1-Score (%)	97.59%	97.70%	99.60%	99.53%
CPU Inference Latency	17.67 ms	31.82 ms	175.36 ms	354.14 ms
Target Deployment Tier	Single-Board Computer	**Small Edge Device**	Edge Gateway / Mobile	Edge Server / Jetson

6.2 Accuracy, Memory, and Computational Cost

Model B achieves 97.70% test accuracy with only 40,175 trainable parameters and a model size of 0.61 MB. Compared with Model A, it reduces the number of trainable parameters by

approximately 84.5% while slightly improving accuracy. This shows that depthwise separable convolutions can help build a smaller model without a major loss in performance.

MobileNetV2 and EfficientNet-B0 achieve higher accuracies of 99.60% and 99.53%, respectively. However, their model sizes are much larger at 26.48 MB and 47.20 MB. Therefore, these pre-trained models offer better accuracy, while Model B is more suitable when storage and memory are limited.

Model B also has the shortest reported training time per epoch at 28.60 seconds. Its CPU inference latency is 31.82 ms, which is higher than Model A's 17.67 ms but considerably lower than the reported latencies of MobileNetV2 and EfficientNet-B0. These results show that a lower parameter count does not always guarantee the fastest inference.

6.3 Deployment Considerations

Model B is a promising option for resource-constrained edge devices because of its small size and competitive accuracy. If its weights are quantized to INT8, they would require approximately 40 KB of storage. However, actual microcontroller deployment also depends on activation memory, runtime requirements, and operator support.

MobileNetV2 may be preferable when higher accuracy is required and the target device has sufficient resources. EfficientNet-B0 is another option when its larger memory and computational requirements can be accommodated. The suitability of each model should be confirmed through testing on the intended hardware.

6.4 Conclusion

Overall, Model B provides a practical balance between accuracy and resource efficiency, achieving 97.70% accuracy with a model size of only 0.61 MB. Although MobileNetV2 and EfficientNet-B0 achieve higher accuracy, they require substantially more storage and have higher reported CPU inference latency. Therefore, Model B is a strong candidate for resource-constrained applications, while pre-trained models are more suitable when higher accuracy is the priority and additional resources are available.

References

[1] M. I. Ahmed, S. M. Mamun, and A. U. Z. Asif, "DCNN-Based Vegetable Image Classification Using Transfer Learning: A Comparative Study," *5th International Conference on Computer, Communication and Signal Processing (ICCCSP)*, 2021.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[4] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning (ICML)*, 2019.

[7] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.